

The Socket Exchange

Assignment 2 — delivery plan, architecture guide and experiment runbook

Revision 2, 5 September 2026 · rebuilt against the revised handout · deadline 10 September 2026, 23:59 IST

What this document is, and what it deliberately is not. It is a delivery plan, an architecture guide, a command-level runbook for all seven mandatory experiments and the bonus, a report skeleton and a viva preparation set. It contains data structure definitions, module boundaries, algorithm outlines and the exact FreeBSD invocations you will need.

It does not contain a finished Exchange Server and client pair ready to drop into `src/`. Twenty percent of this assignment is a viva on code you are expected to have written, and the project is only worth a CV line if the work behind it is yours. So the framing loop, the event loop and the order book appear here as structure, invariants and edge cases rather than as code to paste. Everything that is scaffolding, measurement or investigation is written out in full.

0 What changed in the revised handout

Three substantive changes. The first two tighten validation. The third is a real design constraint that invalidates the obvious implementation of client disconnection.

Change	What it now requires
Explicit numeric ranges	Quantity and price must be positive integers in 1 – 2,147,483,647 ; order IDs in 0 – 2,147,483,647 . A BUY or SELL carrying a value outside the range must be answered with ERROR <reason> and must not be accepted or executed. Range checking is now a stated requirement, not an inference.
OK is pinned down	OK is the response to a successful LOGIN, SUBSCRIBE or UNSUBSCRIBE. A successful CANCEL returns ORDER_CANCELLED <order_id> <i>instead of</i> OK, not in addition to it.
Orders outlive their connection	Closing a Trader Client's connection does not cancel its accepted orders. They stay in the book and may match later. When they do, the trade proceeds normally, the BOUGHT/SOLD notification is simply not sent because nobody is there to receive it, and subscribed Market-Data Clients still receive the TRADE.

The third has teeth. An order can no longer hold a pointer to the connection that created it, because that connection may be freed while the order is still live and matchable. Section 4.4 reworks the order book around that, and it exposes a username-reuse trap worth understanding before you write the code rather than after.

1 Scope and effort model

Stripped of the trading framing, the deliverable is a newline-delimited text protocol server with seven client verbs and seven server messages, an order book with exact-price matching and partial fills, two client programs, seven mandatory experiments with screenshot evidence, a report, and an optional scalability study at 70,000 idle connections. C, C++ or Python; raw sockets only, no asyncio, no libevent, no Boost.Asio.

1.1 Where the marks are

Component	Weight	What actually earns it
Implementation	35%	Protocol conformance, matching correctness, concurrent clients, clean connection teardown
Experiments	40%	Tool-based evidence and correct reasoning. Explicitly stated that a correct implementation alone will not earn full marks here
Report	5%	Clarity, and the concurrency/IO justification in particular

Component	Weight	What actually earns it
Viva	20%	Whether you can explain your own mechanism
Bonus	10%	70k idle connections, measured and analysed

The weighting is the single most important planning input. Implementation is 35% and is the part you are fastest at; experiments and viva together are 60% and are the part that gets under-served when the code runs late. Plan backwards from that: the server exists in order to be instrumented, not the other way round.

1.2 Effort estimate

Workstream	Team-hours	Notes
FreeBSD VM, tooling familiarisation	2–3	sockstat, netstat, procstat, ktrace, tcpdump on loopback
Server core: framing, event loop, lifecycle	7–9	The part that decides your grade in three of the experiments
Order book, matching, ownership, cancel	5–6	Revised upward: order lifetime is now decoupled from connection lifetime
Trader and market-data clients	3–4	Asynchronous server pushes while reading stdin is the fiddly bit
Launchers, README, validation, teardown	2–3	Now includes explicit numeric range checking
Experiments 1–4, 6–8	9–11	About 1.25 h each including capture and write-up
Report	4–5	
Bonus	8–12	Dominated by port exhaustion and memory tuning, not by code
Viva preparation	2	
Total with bonus	42–55	
Total without bonus	34–43	

You have six days now, not seven. As a two-person team that is still comfortable, at roughly five hours each per day. Solo it no longer works with the bonus attached: plan for the seven mandatory experiments and treat any bonus progress as surplus.

2 Six-day schedule

Hours are *per person* for a two-person team working partly in parallel. Each day has an exit condition. The day lost to the revision comes out of the bonus buffer, which is why the bonus now occupies one full day rather than two — and why the go/no-go moves to Monday.

Day	Hrs	Work	Exit condition
Sat 5	6	VM up; record baseline sysctls before touching anything. Skeleton server: listener, accept, kqueue registration, framing buffer, LOGIN only. Partner takes both clients plus tool practice against any listening socket.	python3 experiment.py 1 runs end to end against ./server/run-server
Sun 6	7	Full protocol. Order book with session-scoped ownership, matching, partial fills, CANCEL, SUBSCRIBE/UNSUBSCRIBE and fan-out, role enforcement, numeric range validation, QUIT, every ERROR path.	All seven verbs correct, and the disconnect-with-resting-order test in §4.7 passes
Mon 7	5	Client polish and output buffering. Experiments 1, 2, 3, 4 and 6 — capture and write up in the same sitting.	Five experiments done. Bonus go/no-go decided tonight.

Day	Hrs	Work	Exit condition
Tue 8	5	Experiments 7 and 8, the tcpdump-heavy pair; schedule nothing after them. Then start the client-generation program.	All seven mandatory experiments written up
Wed 9	6	Bonus in full: sysctl tuning, source-address aliasing, climb to 70k, seven measurement rows, thread-per-connection comparison, analysis.	70,000 established connections held; both designs measured
Thu 10	5	Implementation-decisions section, README, zip structure, clean-checkout dry run of all three launchers, submit. Then viva prep.	Submitted by early evening, not at 23:59

2.1 The Monday-night decision point

By Monday night you will know whether the server is solid and whether the first five experiments went cleanly. If both hold, take the bonus: it is the only part of this assignment not already represented somewhere on your CV, and it is the part an interviewer would actually ask about. If either is shaky, drop it and spend Wednesday deepening the seven write-ups instead. Experiments are 40% and the bonus is 10%, so the trade is not close.

3 Architecture decisions

Section 4.7 leaves the concurrency mechanism to you but requires you to defend it. Decide deliberately on Saturday, write the justification down while the reasoning is fresh, and reuse that text in section 8.1 of your report.

3.1 Recommended: single-threaded kqueue event loop

Option	For	Against
Thread per connection	Trivial to write; a blocking <code>recv()</code> per thread reads naturally	8 KB to several MB of stack per connection; will not reach 70k; the shared order book needs locking, which you must then defend in the viva
<code>select()</code>	Portable, simple	<code>FD_SETSIZE</code> caps at 1024 by default; $O(n)$ rescan per iteration; dead on arrival for the bonus
<code>poll()</code>	No fixed descriptor ceiling	Still passes and rescans the whole array on every call; at 70k that cost is measurable and you will have to explain it
<code>kqueue()</code> — recommended	$O(1)$ in the number of idle descriptors; the native FreeBSD mechanism; a single address space, so the order book needs no locking; <code>EV_EOF</code> and writability events map directly onto experiments 6, 7 and 8	More setup code; you must understand level-versus edge-triggered semantics well enough to defend it

`kqueue` also gives the cleanest answer in experiment 4. The harness there sends `LOGIN blocked_client` with no trailing newline and then goes silent. A blocking single-threaded server sits in `recv()` on that descriptor and client 2 stalls; an event loop services client 2 immediately, because the first descriptor simply never becomes readable again.

3.2 A cheap trick that buys you a better experiment 4

Put the blocking behaviour behind a flag — a `--blocking` mode that reads one descriptor to completion before returning to the loop. Two screenshots of the same harness against the same binary, one where client 2 is served in milliseconds and one where it waits, with `procstat -kk` showing the server parked in a receive wait channel in the second, is a substantially stronger deliverable than one screenshot asserting nothing went wrong. Cost: perhaps forty minutes.

3.3 Language

C. Two reasons. First, the bonus measurements are meant to expose per-connection *kernel* cost, and a Python interpreter's per-object overhead sitting on top of that muddies every number in the table — you would end up measuring CPython, not FreeBSD. Second, it is the stronger viva position: questions about socket buffers, descriptor limits and partial writes are far easier to answer when you have been calling the syscalls directly.

Fallback: if by Sunday night the server is not working, switch to Python with the standard socket module and `select.kqueue`. This is permitted — 4.1.2 bars frameworks built on top of sockets, not the socket module itself. Accept that the bonus analysis gets weaker and say so honestly in the report; an accurate account of interpreter overhead confounding the memory column is itself a legitimate finding.

4 Implementation structure

What follows is the shape of the system: modules, state, invariants and the edge cases that break naive implementations. The code inside each function is yours to write.

4.1 Module map

```
src/
main.c          argument parsing, listener setup, signal handling
net.c/.h        listener creation, accept, non-blocking helpers
eventloop.c/.h  kqueue registration, dispatch
conn.c/.h       per-connection state, read/write buffers, framing
proto.c/.h      parse one line -> command struct; validate; format output
book.c/.h       order book, matching, partial fills, cancel
session.c/.h    session table, roles, usernames, subscription sets
mdfeed.c/.h     fan-out of TRADE messages to subscribed sockets
server/run-server      exec wrapper
client/run-trader      exec wrapper
client/run-market-data exec wrapper
```

Two separations carry most of the design. **proto** from **conn**: framing is a byte-stream concern, parsing is a line concern, and experiment 3 exists to check you have not conflated them. And **book** from **conn**: as of the revised handout, orders outlive the connections that created them, so the book must not hold connection pointers at all.

4.2 Connection and session state

```
struct conn {
    int      fd;
    uint64_t session_id;          /* unique per accepted connection,    */
                                   /* never reused for the server's life */
    enum { UNIDENTIFIED, TRADER, MARKET_DATA } role;
    char     username[N];         /* traders only; unique among CONNECTED */

    char     inbuf[IN_CAP];       /* accumulates partial lines          */
    size_t   in_len;

    char     *outbuf;             /* unsent bytes after a partial write */
    size_t   out_len, out_cap;
    int      want_write;         /* whether EVFILT_WRITE is registered */

    uint32_t subscriptions;      /* bitmask over the two instruments    */
};

/* session table: session_id -> struct conn *, or NULL once disconnected.
   Orders reference session_id, never a struct conn *. */
```

The output buffer is not optional. It is what experiment 7 is built to test. When a market-data client stops reading, the server's send buffer for that socket fills, the receiver advertises a zero window, and `send()` on a non-blocking socket returns short or fails with `EAGAIN`. If you assume `send()` writes everything you gave it, you will silently drop market data. Queue the remainder, register `EVFILT_WRITE`, deregister when the queue drains.

4.3 The framing loop

The one invariant: *a read returns bytes, not messages.*

- Append everything `recv()` returned to `inbuf` at offset `in_len`.
- Scan for `\n`. For each one found, the bytes before it are one complete message; hand them to the parser.
- After the last newline, memmove the remainder to the front and update `in_len`. Those bytes are a partial message; keep them and return to the loop.
- A single `recv()` may yield zero complete messages (experiment 3) or several (a pipelining client). Both paths must work.
- `recv()` returning 0 is an orderly close from the peer; `-1` with `ECONNRESET` means the peer sent RST. These are different events, and experiment 6 asks you to distinguish them.
- If `in_len` reaches `IN_CAP` with no newline, the peer is sending an unbounded line. Decide the policy now — error and disconnect is defensible; growing without limit is a resource leak an examiner may ask about.

4.4 Order book and order ownership

The matching rule is much weaker than a real CLOB: same instrument, opposite side, *exactly equal price*. No price-time priority, no crossing at better prices, no balances. Do not over-build it — marks here come from correctness, not sophistication.

What is subtle, and new in the revised handout, is lifetime. An accepted order survives its creator's disconnection, stays matchable, and can trade long after the trader is gone. So the book stores an owning **session id**, and notification is a lookup that is allowed to fail:

```
Data: two instruments x two sides -> four collections of resting orders,
      each keyed by exact price. A price bucket holds orders with
      remaining_qty > 0.
      order_id -> { owner_session, instrument, side, price, remaining }
```

```
notify(session_id, message):
    c = session_table[session_id]
    if c == NULL: return          # disconnected: drop it, silently, per spec
    enqueue message on c's output buffer
```

```
on_new_order(conn, side, instrument, qty, price):
    validate instrument, qty, price      # see 4.5 - ERROR and return if bad
    id = next_order_id++
    send ORDER_ACCEPTED <id> to conn     # BEFORE any matching
    remaining = qty
    while remaining > 0 and a resting opposite order exists at exactly price:
        peer = pick one such resting order
        traded = min(remaining, peer.remaining)
        peer.remaining -= traded
        remaining -= traded
        notify(buyer_session, "BOUGHT <instrument> <traded> <price>")
        notify(seller_session, "SOLD <instrument> <traded> <price>")
        broadcast to every subscriber of <instrument>:
            "TRADE <instrument> <traded> <price>"
        if peer.remaining == 0: remove peer from book and id map
    if remaining > 0: rest the order at <price> with owner = conn.session_id
```

```
on_disconnect(conn):
    session_table[conn.session_id] = NULL    # orders stay in the book
    release conn.username                    # free for a future LOGIN
    free conn
```

4.4.1 The username-reuse trap

Ownership must be scoped to the *session*, not the username. Consider: alice logs in, submits an order, disconnects. Her order is still resting. Her username is now free, because uniqueness is only required among currently connected traders. A new client connects and sends `LOGIN alice` — legally. If your book keyed ownership by username, this new alice would start receiving `BOUGHT` notifications for an order she never placed, and could `CANCEL` it. Both are wrong: the spec says the notification is not sent, because *the originating client* is no longer connected. A monotonic session id that is never reused makes this fall out correctly with no special case.

This is also a very likely viva question, and a good one to raise yourself in the implementation-decisions section — it shows you read the ownership rule rather than pattern-matching it.

4.5 Numeric validation

Now an explicit requirement rather than an inference. Quantity and price must lie in 1–2,147,483,647; order ids in 0–2,147,483,647. Anything outside gets ERROR <reason> and the order must not be accepted or executed — note that this means the check runs *before* you allocate an order id, so a rejected order consumes nothing.

- Do not parse into a 32-bit int and then range-check; the overflow has already happened. Parse into a 64-bit type (or use `strtol` with `errno` and an end pointer) and check the bounds afterwards.
- Reject anything that is not wholly digits: empty fields, embedded signs, decimal points, trailing garbage such as `100abc`. `strtol` stops at the first non-digit and reports success, so check that the end pointer reached the end of the token.
- 2,147,483,647 is `INT32_MAX`. A matched trade quantity is the minimum of two remaining quantities, so it cannot overflow — but if you ever sum quantities or multiply quantity by price for any reason, that product does not fit in 32 bits. Do not compute it unless you need it.
- Same discipline for CANCEL <order_id>: a malformed or out-of-range id is an ERROR, not a lookup miss.

4.6 Edge cases that will be tested

Case	Required behaviour
Trader sends SUBSCRIBE, or MD client sends BUY	ERROR. Role enforcement is explicit in 4.5 of the handout.
Duplicate username among <i>connected</i> traders	ERROR. Uniqueness holds only over currently connected clients, so the name must be released on disconnect.
Quantity or price of 0, negative, decimal, or > 2,147,483,647	ERROR <reason>; the order is neither accepted nor executed and no order id is consumed.
Unknown instrument	ERROR. Only JNST and IMCT exist.
Successful LOGIN / SUBSCRIBE / UNSUBSCRIBE	OK.
Successful CANCEL	ORDER_CANCELLED <order_id> — <i>instead of</i> OK, not as well as.
CANCEL of an unknown, fully filled, or another session's order	ERROR. Ownership is checked against session id, not username.
CANCEL of a partially filled order	Permitted; cancel the remaining quantity only.
Trader disconnects holding resting orders	Orders stay in the book and remain matchable. Release the username, null the session entry, free the connection. Do not touch the orders.
A resting order matches after its owner disconnected	The trade happens normally. BOUGHT/SOLD is skipped for the absent party — and still delivered to the present counterparty. Subscribed Market-Data Clients still get TRADE.
Same username logs in again while old orders rest	The new session must not receive fills for, or be able to cancel, the old session's orders.
Client disconnects mid-write from the server	A write to a reset socket raises SIGPIPE and will kill your server. Ignore SIGPIPE or use <code>MSG_NOSIGNAL</code> . This is experiment 8.
Commands before LOGIN	Decide and document. Rejecting until identified is defensible; whatever you pick, be consistent and say so in the report.
Empty line, unknown verb, trailing whitespace, <code>\r\n</code> endings	Do not crash. Someone will paste something malformed with <code>nc</code> .

4.7 The one test to run before you touch the experiments

The disconnection rule is the easiest thing in the revised spec to get wrong and the hardest to notice, because nothing crashes. Drive it by hand once, on Sunday, before you start capturing anything:

```
term 1:  ./server/run-server 127.0.0.1 5000
term 2:  ./client/run-market-data 127.0.0.1 5000    -> SUBSCRIBE JNST
term 3:  ./client/run-trader 127.0.0.1 5000        -> LOGIN alice
                                                -> BUY JNST 100 238
                                                -> (note order id)

        then kill term 3 outright
term 4:  ./client/run-trader 127.0.0.1 5000        -> LOGIN bob
                                                -> SELL JNST 60 238

expect:  bob receives SOLD JNST 60 238
        term 2 receives TRADE JNST 60 238
        no BOUGHT is sent anywhere, and the server does not crash
        40 units of alice's order still rest in the book
```

```
then:    reconnect as alice and try to CANCEL the old order id
expect:  ERROR - it belongs to a dead session, not to this one
```

4.8 Launchers

Three files, all executable, all using `exec` so the launcher process is replaced rather than left as a parent. This matters more than it looks: the harness terminates the process it spawned, and if your launcher is still sitting in front of the real server, the server survives the kill and the next experiment fails on a bound port.

```
#!/bin/sh
exec ./exchange_server "$@"
```

`chmod +x` all three and confirm the mode survives the zip. A structure failure costs 10% and a resubmission.

5 Protocol quick reference

Direction	Message	Applies to
client → server	LOGIN <username>	Trader
	BUY <instrument> <qty> <price>	Trader
	SELL <instrument> <qty> <price>	Trader
	CANCEL <order_id>	Trader
	SUBSCRIBE <instrument>	Market-data
	UNSUBSCRIBE <instrument>	Market-data
	QUIT	Both
server → client	OK	LOGIN, SUBSCRIBE, UNSUBSCRIBE only
	ERROR <reason>	Both
	ORDER_ACCEPTED <order_id>	Trader
	ORDER_CANCELLED <order_id>	Trader; replaces OK for CANCEL
	BOUGHT <instrument> <qty> <price>	Trader, own fills only
	SOLD <instrument> <qty> <price>	Trader, own fills only
	TRADE <instrument> <qty> <price>	Market-data, subscribed instruments

Every message is one line terminated by `\n`. Instruments: JNST, IMCT. Quantity and price are integers in 1–2,147,483,647; order ids in 0–2,147,483,647, unique for the server's lifetime. TRADE deliberately does not reveal which side the recipient was on — that is what BOUGHT and SOLD are for, and conflating them is a protocol error.

6 Experiment runbook

Forty percent of the grade. The handout is explicit that a correct implementation alone will not earn full marks here — what is being assessed is whether you reached the answer through the tools. Two habits carry most of the difference:

- **Write the answer the same hour you capture the screenshot.** The reasoning is fresh, and you will notice when the evidence does not actually support the claim while you can still re-run.
- **Every screenshot must be self-identifying.** The command, the output, and enough context (pid, port, connection count) to tell what was running. A cropped netstat line proves nothing. Leave the shell prompt visible.

6.0 Tool cheat sheet

sockstat -4 -l	listening sockets, with pid and process
sockstat -4 -c	connected sockets only
sockstat -4 grep exchange	everything your server holds
netstat -an -p tcp	states, Recv-Q / Send-Q per connection
netstat -Aan -p tcp	same, with PCB addresses
netstat -m	mbuf and cluster usage
netstat -s -p tcp	protocol counters (retransmits, probes)
procstat -f PID	the process's open descriptors
procstat -kk PID	kernel stack of every thread: what it is blocked in, by name
procstat -v PID	VM map, for resident memory detail
ktrace -i -p PID ; kdump -f ktrace.out	syscall-level trace: the actual recv() return values
tcpdump -i lo0 -n -S -vv 'tcp port 5000'	absolute sequence numbers, loopback
ps -o pid,rss,vsz,%cpu -p PID	process memory and CPU
vmstat -z egrep 'socket tcpcb inpcb'	kernel zone allocations per object type

Run tcpdump on lo0 — everything in this assignment is loopback traffic and it will not appear on em0. Start the capture *before* the harness, and use -w to a file so you can re-read it without re-running the experiment.

6.1 Experiment 1 — listening vs connected sockets

Question: after a client connects, how does the listening socket differ from the connection socket?

```
python3 experiment.py 1          # leave running
sockstat -4 -l | grep -i exchange
sockstat -4 | grep -i exchange
netstat -an -p tcp | egrep 'LISTEN|ESTABLISHED'
procstat -f `pgrep exchange_server`
```

What the answer turns on: the listening socket is bound to a local address with no foreign endpoint and sits in LISTEN; it is not a connection and carries no data. The connected socket is a distinct descriptor returned by accept(), identified by the full four-tuple, in ESTABLISHED. Note that both share the same local port — that is the detail worth calling out, because it is what makes the four-tuple, not the port, the connection's identity. The procstat -f output showing both descriptors held by one process makes the point cleanly.

6.2 Experiment 2 — connection states over a lifetime

Question: how does the TCP state change over the connection's life, and what causes each change?

```
tcpdump -i lo0 -n -S 'tcp port 5000' -w exp2.pcap &
python3 experiment.py 2
# at each pause the harness gives you:
netstat -an -p tcp | grep 5000
# afterwards:
tcpdump -r exp2.pcap -n -S
```

Capture the same netstat at three moments: after connect, while idle, and after one side closes. You are looking for LISTEN, ESTABLISHED, and then the teardown sequence — which side goes to FIN_WAIT and which to CLOSE_WAIT tells you who closed first. If you see TIME_WAIT, explain why it exists and roughly how

long it lasts; that is a very likely viva follow-up.

6.3 Experiment 3 — TCP is a byte stream

Question: does the server receive one application message as one unit?

The harness deliberately dribbles one valid message out in pieces. The strongest evidence is syscall-level, not packet-level, because the question is about what `recv()` returned to your program:

```
python3 experiment.py 3 &
ktrace -i -p `pgrep exchange_server`
# let it run, then
kdump -f ktrace.out | egrep -A1 'recvfrom|read'
```

You want the trace showing several returns of a few bytes each, none of which is a complete line, followed by your parser firing exactly once when the newline finally arrives. A debug line on `stderr` printing every `recv()` return value alongside the buffer length is an acceptable and much simpler alternative — the handout explicitly permits modifying your implementation to make the behaviour observable. Doing both is better.

6.4 Experiment 4 — one idle client must not stall others

Question: with client 1 connected but silent, can the server still service client 2? Identify the operation that decides it.

```
python3 experiment.py 4 &
procstat -kk `pgrep exchange_server`      # what is the thread blocked in?
sockstat -4 | grep -i exchange             # both connections present?
netstat -an -p tcp | grep 5000             # both ESTABLISHED?
```

The harness prints client 2's elapsed time; that number is your headline evidence and `procstat -kk` is your explanation. In the event-loop build the thread is parked in `kevent` and client 2 is answered in milliseconds. If you built the `--blocking` variant suggested in §3.2, run it both ways: the contrast between a thread waiting in a socket receive and one waiting in the event queue, with the two elapsed times beside them, answers the question far more convincingly than either run alone.

6.5 Experiment 5 — readiness (optional)

Five connections, only three of which send. Cheap to do once the machinery from experiment 4 is in place: log which descriptors `kevent` reported as ready in each wakeup, and corroborate with `netstat -an Recv-Q`, which will be non-zero only for the sockets with unread data. Thirty minutes for a cleaner story about your I/O model. Take it if experiments 1–4 finished on schedule; skip it without regret if not.

6.6 Experiment 6 — FIN vs RST

Question: how does abrupt termination differ from orderly shutdown, at the packet level and at the server's socket?

```
tcpdump -i lo0 -n -S -vv 'tcp port 5000' -w exp6.pcap &
python3 experiment.py 6
# part A is a shutdown(SHUT_WR); part B sets SO_LINGER {1,0} to force RST
netstat -an -p tcp | grep 5000      # run during BOTH parts
tcpdump -r exp6.pcap -n -S | egrep 'Flags \[F\]|Flags \[R\]
```

The two things to pair up: on the wire, a FIN flag versus an RST flag. At the socket, an orderly FIN makes `recv()` return 0 and leaves the server side in `CLOSE_WAIT` until it closes; an RST makes a pending or subsequent operation fail with `ECONNRESET` and destroys the connection state immediately with no `TIME_WAIT`. Say explicitly what `SO_LINGER` with a zero timeout does, since the harness uses it to manufacture the RST.

6.7 Experiment 7 — backpressure and the slow receiver

Question: how does the connection to the slow client behave, and does it eventually affect the other clients?

```
tcpdump -i lo0 -n -S -vv 'tcp port 5000' -w exp7.pcap &
python3 experiment.py 7 &
# sample repeatedly while trades are being generated:
while :; do netstat -an -p tcp | grep 5000; echo ---; sleep 2; done
netstat -s -p tcp | egrep -i 'window probe|persist'
netstat -m
```

This is the richest experiment in the set and the one most worth spending extra time on. Watch the advertised window on the slow client's connection shrink toward zero in the capture, then look for zero-window probes. In `netstat -an`, the slow client's Recv-Q fills and the server's Send-Q for that socket backs up; compare side by side with the healthy client's connection, which should stay near zero throughout.

The second half of the question is the one people miss. Whether the slow client harms the others is a property of *your design*, not of TCP: with a per-connection output queue and `EVFILT_WRITE`, the other clients are unaffected and only the slow socket's queue grows — and you should then say what bounds that queue, because unbounded growth is a memory exhaustion vector. If instead you call a blocking `send()` in the broadcast path, the whole server stalls behind one slow reader. State which of these your server does and prove it.

6.8 Experiment 8 — unexpected disconnection

Question: what happens to the connection when a client disappears mid-conversation, and how does the server detect it?

```
tcpdump -i lo0 -n -S -vv 'tcp port 5000' -w exp8.pcap &
python3 experiment.py 8
netstat -an -p tcp | grep 5000          # before and after the kill
tcpdump -r exp8.pcap -n -S
```

Trace the sequence carefully, because it is subtler than "the client died so the server got an error". Killing the process makes the kernel close its sockets, which puts a FIN on the wire — the client is gone but the connection is closed politely. The server, still generating market data, keeps sending. Those segments now arrive at a host with no matching socket, which answers with RST, and the server's *next* write fails. Two consequences worth stating: detection is write-driven, so a server that only ever reads would not notice at all until much later; and the failing write will raise `SIGPIPE` and kill your process unless you handled it. If the client you killed was a *trader* rather than a market-data client, note the revised rule from §4.4: its resting orders survive, so a server that tears down order state on disconnect is both wrong and likely to look correct in this experiment. Compare against the surviving market-data connection in the same screenshot.

7 Bonus — 70,000 idle connections

Ten percent, and the only part of this assignment not already represented somewhere on your CV. It is also the part most likely to consume a day you did not budget, for reasons that have nothing to do with writing code. With the schedule now compressed to a single day, read §7.1 before Wednesday morning rather than discovering it at connection 55,000.

7.1 The obstacle nobody plans for: ephemeral ports

A TCP connection is identified by the four-tuple (source address, source port, destination address, destination port). Your load generator connects from one address to one address and one port, so three of the four are fixed and the only thing that can vary is the source port — a 16-bit field. FreeBSD's default ephemeral range is narrower still:

```
sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
# defaults are roughly 10000 and 65535 -> about 55,000 usable ports
```

Even widened to its practical maximum, a single source address cannot reach 70,000. Two ways out; pick one deliberately and explain it in the report, because it is the most interesting thing in the whole bonus:

- **Alias additional loopback addresses** and bind the source before connecting. Two source addresses double the space. The cleaner answer, because it demonstrates you understand what identifies a connection.
- **Listen on several ports** and spread connections across them. Equivalent arithmetic, slightly less elegant, and it changes your server's shape.

```
# option 1: extra source addresses
ifconfig lo0 alias 127.0.0.2/32
ifconfig lo0 alias 127.0.0.3/32
sysctl net.inet.ip.portrange.first=1024
sysctl net.inet.ip.portrange.last=65535
```

```
# in the generator, before connect():
# bind(fd, one of 127.0.0.2 / 127.0.0.3, port 0)
# then connect to 127.0.0.1:5000
```

7.2 Limits to raise, and baselines to record first

Record every one of these *before* you change anything. The before-and-after is part of the analysis, and an untuned baseline is what makes the tuned numbers mean something.

Knob	Why it binds
kern.maxfiles	System-wide descriptor ceiling. 70k sockets plus everything else will exceed the default.
kern.maxfilesperproc	Per-process ceiling; your server alone needs more than 70k.
ulimit -n (login class)	The soft limit your shell hands the process. Raising the sysctl without raising this achieves nothing.
kern.ipc.maxsockets	Ceiling on socket structures. Distinct from descriptors.
kern.ipc.nmbclusters	mbuf clusters backing socket buffers. Watch netstat -m.
net.inet.tcp.sendspace / recvspace	Per-socket buffer sizing. At 70k connections the default sizing is the difference between comfortable and impossible.
kern.ipc.maxsockbuf	Upper bound on any single socket buffer.
VM RAM	The handout suggests 2 GB for the main assignment. Do not assume that survives the bonus; give the VM more and note that you had to.

7.3 Measurement commands, per table row

```
# take these WHILE the target number of connections is established and idle
SRV=`pgrep exchange_server`
```

```
# established connection count (the row label - prove it, don't assert it)
netstat -an -p tcp | grep -c ESTABLISHED
```

```
# server memory
ps -o pid,rss,vsz -p $SRV
procstat -v $SRV | tail -1
```

```
# server CPU
ps -o pid,%cpu,time -p $SRV
top -b -n 1 | head -12
```

```
# descriptors held by the server, and system-wide
procstat -f $SRV | wc -l
sysctl kern.openfiles kern.maxfiles
```

```
# socket buffer / mbuf usage against its limit
netstat -m
vmstat -z | egrep 'socket|tcp|inpcb|mbuf'
sysctl kern.ipc.maxsockets kern.ipc.numopensockets
```

Script this as one shell function taking the connection count as an argument and dumping everything under a header. You need seven rows, then seven more for the comparison design; doing that by hand fourteen times is how a single-day bonus becomes a two-day bonus. One screenshot per required row (10k, 40k, 70k) showing the count and the measurements together.

7.4 The analysis is the actual deliverable

The table is evidence; the five questions after it are what is graded. Do not predict the answers before you measure — find out which resource actually binds first on your VM and let the numbers drive the argument. Things worth attending to as you go:

- Idle connections are cheap in a way active ones are not. FreeBSD grows socket buffers on demand, so an idle connection costs its kernel structures and little else — which is why the resource that binds first may not be the one you expected.
- Compare kernel memory against process RSS deliberately. A single-threaded event loop holding 70,000 descriptors has a small user-space footprint and a large kernel one; reporting only ps output misses most of the cost, and an examiner will notice.
- For question 4 the comparison is the point. A thread-per-connection build with even a modest stack per thread will not survive four figures, let alone five. Measure where it actually falls over rather than asserting that it would — that measured failure point is the single most quotable number you will produce in this assignment.
- Question 5 asks you to quantify the trade-off after changing something. Shrinking sendspace and recvspace and re-measuring is the cheapest way to earn it — and it has a real cost in market-data throughput, which you should acknowledge rather than present as free.

8 Report skeleton

- 1 System overview ~1 page
protocol, components, how to run
- 2 Implementation decisions ~1 page [5% here]
 - 2.1 concurrency / IO mechanism chosen
 - 2.2 why - what was rejected and on what grounds
 - 2.3 framing strategy
 - 2.4 output buffering and backpressure policy
 - 2.5 order ownership and lifetime across disconnection
 - 2.6 input validation and numeric range enforcement
- 3 Experiments ~1 page each [40% here]
for each of 1,2,3,4,(5),6,7,8:
question | approach | commands and tools | screenshots | answer
... in that order, every time
- 4 Bonus ~3 pages [10% here]
 - 4.1 client generation program and how it reaches 70k
 - 4.2 system limits raised, with before/after values
 - 4.3 measurement table (7 rows x 6 columns)
 - 4.4 screenshots at 10k / 40k / 70k
 - 4.5 analysis: the five questions
 - 4.6 alternative design measured and compared
- 5 Build and run instructions (mirror the README)

Section 2.5 is new to this revision and is worth writing properly: it is the one place you can show that you noticed orders and connections have independent lifetimes, which is the least obvious requirement in the whole specification.

9 Viva preparation

Twenty percent, on your own code. Spend the last hour on Thursday going through these out loud; any answer that takes more than about thirty seconds to assemble is the one you will be asked.

- Why kqueue rather than poll or threads? What would change if you had 50 clients instead of 70,000?
- Level-triggered versus edge-triggered: which are you using, and what breaks if you drain a socket incompletely?
- Walk through what happens when a single recv() returns two and a half messages.
- send() returns fewer bytes than you asked for. What do you do, and what happens if you ignore it?
- A trader disconnects with a resting order. Who owns that order now, and what happens when it matches?
- The same username reconnects while the old session's orders are still in the book. Why can the new session not cancel them, and what in your data model enforces that?

- A client sends a price of 4,000,000,000. Where exactly does that get rejected, and what would have happened if you had parsed it into an int first?
- A client is killed. Trace exactly how and when your server finds out.
- Why does the listening socket share a port with every connection it accepted, and why is that not a conflict?
- What is TIME_WAIT for, which side enters it, and why might it matter to a server restarting on the same port?
- What did you have to change to reach 70,000 connections, and which resource ran out first?
- Where is the unbounded resource in your design, and what would you bound first if this had to run in production?

10 Submission checklist

Run this on Thursday afternoon from a fresh extraction of the zip in a clean directory — not from your working tree, where a stray file outside the submission will make a broken build look fine.

```
[ ] A2_<ROLL_A>_<ROLL_B>.zip    roll numbers alphabetically sorted,
                                exactly as they appear on LMS, capitals,
                                no spaces or hyphens
[ ] server/run-server           executable, uses exec, forwards "$@"
[ ] client/run-trader           executable, uses exec, forwards "$@"
[ ] client/run-market-data      executable, uses exec, forwards "$@"
[ ] src/                        all sources; builds with nothing outside the zip
[ ] README.md                  language, runtime, build commands, how to
                                start server and each client, any config
[ ] report.pdf                  all seven experiments + implementation
                                decisions (+ bonus)
[ ] executable bits survived the zip (unzip elsewhere and check)
[ ] all 8 harness runs pass against the extracted copy
[ ] the 4.7 disconnect test still passes against the extracted copy
[ ] only one team member uploads
```

Late penalty is 20% per day for three days, then the portal closes. A zip that fails the automated structure check costs 10% and one resubmission — a genuinely painful way to lose marks on a filename, so have your partner verify the name against LMS independently rather than checking it twice yourself.

On the CV question: if you complete the bonus, the line to write is about the connection-scalability study — the measured bottleneck, the two I/O designs compared, the number where the thread-per-connection build failed. Not "built a TCP exchange server", which duplicates the matching-engine platform already on your page. And write it only if the code under it is yours, because the follow-up question in an interview is always one layer below whatever the bullet claims.